DL3

Convolutional neural network (CNN) (Any One from the following)

 Use any dataset of plant disease and design a plant disease detection system using CNN.

 Use MNIST Fashion Dataset and create a classifier to classify fashion clothing into categories.

```
# Import required libraries
import tensorflow as tf  # Deep learning framework
from tensorflow.keras.models import Sequential  # To build sequential CNN model
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout  # CNN layers
from tensorflow.keras.preprocessing.image import ImageDataGenerator  # For image preprocessing
import matplotlib.pyplot as plt  # For plotting graphs

# ---------- DATA PREPROCESSING ----------

# Create training data generator with augmentation
train_datagen = ImageDataGenerator(
    rescale=1./255,          # Normalize pixel values (0-255 → 0-1)
    shear_range=0.2,         # Apply shear transformation (tilt images)
    zoom_range=0.2,          # Zoom images randomly
    horizontal_flip=True     # Flip images horizontally
)

# Create testing data generator (only normalization)
```

```
test_datagen = ImageDataGenerator(rescale=1./255)

# Load training dataset from directory
training_set = train_datagen.flow_from_directory(
    'dataset/train',         # Path to training dataset folder
    target_size=(128, 128),  # Resize images to 128x128
    batch_size=32,           # Number of images per batch
    class_mode='categorical' # Multi-class classification
)

# Load testing dataset from directory
test_set = test_datagen.flow_from_directory(
    'dataset/test',          # Path to testing dataset folder
    target_size=(128, 128),
    batch_size=32,
    class_mode='categorical'
)

# ---------- BUILDING CNN MODEL ----------

# Initialize CNN model
cnn = Sequential()

# First Convolution Layer
cnn.add(Conv2D(
    filters=32,              # Number of filters (feature detectors)
    kernel_size=3,           # Size of filter (3x3)
    activation='relu',       # Activation function
    input_shape=[128, 128, 3] # Input image shape (RGB image)
```

```
))

# First Pooling Layer
cnn.add(MaxPooling2D(
    pool_size=2,            # Pooling size (2x2)
    strides=2               # Step size
))

# Second Convolution Layer
cnn.add(Conv2D(
    filters=32,
    kernel_size=3,
    activation='relu'
))

# Second Pooling Layer
cnn.add(MaxPooling2D(
    pool_size=2,
    strides=2
))

# Flatten layer (convert 2D feature maps into 1D vector)
cnn.add(Flatten())

# Fully connected layer
cnn.add(Dense(
    units=128,              # Number of neurons
    activation='relu'
))
```

```
# Dropout layer to prevent overfitting
cnn.add(Dropout(0.5))       # Randomly disables 50% neurons during training

# Output layer
cnn.add(Dense(
    units=training_set.num_classes, # Number of output classes
    activation='softmax'            # For multi-class classification
))

# -------------------- COMPILING MODEL --------------------

cnn.compile(
    optimizer='adam',               # Optimizer to update weights
    loss='categorical_crossentropy', # Loss function for multi-class
    metrics=['accuracy']            # Evaluation metric
)

# -------------------- TRAINING MODEL --------------------

history = cnn.fit(
    x=training_set,           # Training data
    validation_data=test_set, # Validation data
    epochs=10                 # Number of iterations
)

# -------------------- VISUALIZATION --------------------

# Plot training and validation accuracy
```

```
plt.plot(history.history['accuracy'], label='train accuracy')
plt.plot(history.history['val_accuracy'], label='validation accuracy')
plt.legend()
plt.title('Accuracy')
plt.show()

# Plot training and validation loss
plt.plot(history.history['loss'], label='train loss')
plt.plot(history.history['val_loss'], label='validation loss')
plt.legend()
plt.title('Loss')
plt.show()

# ---------- SAVE MODEL ----------

cnn.save('plant_disease_model.h5')  # Save trained model to file
```

🔹 1. Import Libraries

```
import tensorflow as tf
```

👉 Sir, I am importing TensorFlow which is used for building deep learning models.

```
from tensorflow.keras.models import Sequential
```

👉 Sequential is used to create a model layer-by-layer in sequence.

```
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout
```

👉 These are different layers of CNN:

- Conv2D → feature extraction
- MaxPooling → reduce size
- Flatten → convert to 1D
- Dense → fully connected layer
- Dropout → prevent overfitting

```
from tensorflow.keras.preprocessing.image import ImageDataGenerator
```

👉 Used to preprocess and augment images.

```
import matplotlib.pyplot as plt
```

👉 Used to plot graphs like accuracy and loss.

🔹 2. Data Preprocessing

```
train_datagen = ImageDataGenerator(
    rescale=1./255,
    shear_range=0.2,
    zoom_range=0.2,
    horizontal_flip=True
)
```

👉 Sir, here I am preparing training data:

- rescale → normalize pixel values (0–255 → 0–1)
- shear_range → tilt images
- zoom_range → zoom images
- horizontal_flip → flip images

👉 This is called **data augmentation** to increase dataset variety.

---

test_datagen = ImageDataGenerator(rescale=1./255)

👉 For test data, only normalization is done (no augmentation).

```
training_set = train_datagen.flow_from_directory(
    'dataset/train',
    target_size=(128, 128),
    batch_size=32,
    class_mode='categorical'
)
```

👉 Sir, this loads training images:

- Path = dataset/train
- Resize to 128×128
- Batch size = 32 images at a time
- Categorical → multi-class classification

```
test_set = test_datagen.flow_from_directory(
    'dataset/test',
    target_size=(128, 128),
    batch_size=32,
    class_mode='categorical'
)
```

👉 Same for testing dataset.

🔹 3. Building CNN Model

cnn = Sequential()

👉 Sir, I am initializing the CNN model.

🔸 First Convolution Layer

cnn.add(Conv2D(filters=32, kernel_size=3, activation='relu', input_shape=[128, 128, 3]))

👉 This layer:

- Extracts features from image
- 32 filters → detect patterns like edges
- Kernel size 3×3
- ReLU → removes negative values
- Input shape = 128×128 RGB image

---

cnn.add(MaxPooling2D(pool_size=2, strides=2))

👉 Reduces image size (downsampling) to make computation faster.

🔸 **Second Convolution Layer**

cnn.add(Conv2D(filters=32, kernel_size=3, activation='relu'))

👉 Again extracts deeper features.

---

cnn.add(MaxPooling2D(pool_size=2, strides=2))

👉 Again reduces size.

🔹 **4. Flattening**

cnn.add(Flatten())

👉 Converts 2D feature maps into 1D vector so we can give it to Dense layer.

🔹 **5. Fully Connected Layers**

cnn.add(Dense(units=128, activation='relu'))

👉 Hidden layer with 128 neurons.

---

cnn.add(Dropout(0.5))

👉 Randomly drops 50% neurons to prevent overfitting.

```
cnn.add(Dense(units=training_set.num_classes, activation='softmax'))
```

👉 Output layer:

- Number of neurons = number of classes
- Softmax → gives probability for each class

🔹 6. Compile Model

```
cnn.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
```

👉 Sir:

- Adam → optimizer to update weights
- Loss → categorical crossentropy for multi-class
- Metric → accuracy

🔹 7. Training Model

```
history = cnn.fit(
    x=training_set,
    validation_data=test_set,
    epochs=10
)
```

👉 Model is trained:

- Using training data
- Tested on validation data
- Runs for 10 epochs

🔹 8. Plot Accuracy

```
plt.plot(history.history['accuracy'], label='train accuracy')
plt.plot(history.history['val_accuracy'], label='validation accuracy')
plt.legend()
plt.title('Accuracy')
plt.show()
```

👉 Shows how accuracy improves during training.

🔹 9. Plot Loss

```
plt.plot(history.history['loss'], label='train loss')
plt.plot(history.history['val_loss'], label='validation loss')
plt.legend()
plt.title('Loss')
plt.show()
```

👉 Shows how error decreases.

🔹 10. Save Model

```
cnn.save('plant_disease_model.h5')
```

👉 Saves trained model so we can use it later.

🎯 FINAL VIVA SUMMARY (VERY IMPORTANT)

👉 “Sir, this project uses CNN to classify plant diseases.
Images are preprocessed and augmented, then passed through convolution and pooling layers for feature extraction.
After flattening, fully connected layers perform classification using softmax.
Model is trained using Adam optimizer and evaluated using accuracy, and finally saved for future prediction.”

🔥 DRY RUN WITH CODE (STEP-BY-STEP)

🔹 STEP 1: Start Program

```
import tensorflow as tf
...
```

👉 Program starts
👉 All libraries are loaded into memory

🔹 STEP 2: Data Generator Creation

```
train_datagen = ImageDataGenerator(...)
```

👉 No images loaded yet
👉 Just **rules defined**:

- Normalize
- Flip
- Zoom
- Shear

```
test_datagen = ImageDataGenerator(rescale=1./255)
```

👉 Only normalization rule stored

🔹 STEP 3: Load Dataset

```
training_set = train_datagen.flow_from_directory(...)
```

👉 Now actual action happens:

Assume:

- You have 1000 images
- 3 classes → Healthy, Rust, Powdery

👉 Internally it does:

Batch 1 → 32 images
Batch 2 → 32 images
...

👉 Each image:

- Resized → 128×128
- Converted → pixel range [0,1]

👉 Labels converted:

Healthy → [1,0,0]
Rust → [0,1,0]
Powdery → [0,0,1]

---

test_set = test_datagen.flow_from_directory(...)

👉 Same process for test data

🔹 STEP 4: Model Creation

cnn = Sequential()

👉 Empty model created
👉 No layers yet

🔹 STEP 5: First Conv Layer

cnn.add(Conv2D(...))

👉 Input: One image (128×128×3)

👉 Operation:

- Apply 32 filters (3×3)

👉 Output:

128×128×3 → 126×126×32

👉 Why?

- Kernel reduces size slightly
- Depth becomes 32 (filters)

🔹 **STEP 6: MaxPooling**

cnn.add(MaxPooling2D(...))

👉 Input: 126×126×32
👉 Output:

63×63×32

👉 Reduces size → faster processing

🔹 **STEP 7: Second Conv Layer**

cnn.add(Conv2D(...))

👉 Input: 63×63×32
👉 Output:

61×61×32

👉 Extract deeper features

🔹 **STEP 8: Second Pooling**

cnn.add(MaxPooling2D(...))

👉 Output:

30×30×32

🔹 **STEP 9: Flatten**

cnn.add(Flatten())

👉 Converts:

30×30×32 → 28800 neurons

👉 Now becomes 1D vector

🔹 **STEP 10: Dense Layer**

cnn.add(Dense(128))

👉 Input: 28800
👉 Output: 128 neurons

👉 Learns patterns

🔹 **STEP 11: Dropout**

cnn.add(Dropout(0.5))

👉 Randomly disables 50% neurons
👉 Prevents overfitting

🔹 **STEP 12: Output Layer**

cnn.add(Dense(num_classes))

👉 Suppose 3 classes → output:

[0.1, 0.7, 0.2]

👉 Meaning:

- 10% Healthy
- 70% Rust ✅ (predicted)
- 20% Powdery

🔹 **STEP 13: Compile**

cnn.compile(...)

👉 Model ready for training
👉 Defines:

- How to update weights (Adam)
- How to calculate error (loss)

🔹 **STEP 14: Training Starts**

cnn.fit(...)

👉 Loop starts:

🔁 **For each epoch (10 times):**

For each batch:

1. Take 32 images
2. Forward pass:
    - Conv → Pool → Conv → Pool → Dense
3. Get prediction
4. Compare with actual label
5. Calculate loss
6. Backpropagation
7. Update weights

🔹 **STEP 15: Example Flow (Single Image)**

👉 Input image → leaf with disease

Step 1: CNN detects edges
Step 2: CNN detects shapes/spots
Step 3: CNN detects disease pattern
Step 4: Output = class probability

🔹 **STEP 16: Accuracy & Loss Stored**

history.history

👉 Example:

accuracy → [0.60, 0.75, 0.85, ...]
loss → [1.2, 0.8, 0.5, ...]

🔹 **STEP 17: Plot Graph**

plt.plot(...)

👉 Graph shows:

- Accuracy increasing ✅
- Loss decreasing ✅

🔹 STEP 18: Save Model

```
cnn.save('plant_disease_model.h5')
```

👉 Final trained model stored
👉 Can reuse later without retraining

🎯 FINAL DRY RUN SUMMARY (FOR VIVA)

👉 Say this confidently:

“Sir, first images are loaded and preprocessed using ImageDataGenerator.
Then CNN model is built using convolution and pooling layers to extract features.
Flatten layer converts data into 1D and Dense layers perform classification.
During training, images are passed in batches, predictions are made, loss is calculated, and weights are updated using backpropagation.
Finally, accuracy and loss are plotted and the trained model is saved.”

🔥 TOP VIVA QUESTIONS (WITH PERFECT ANSWERS)

🔹 1. What is your project?

👉 **Answer:**
“Sir, my project is a plant disease detection system using Convolutional Neural Network. It takes an image of a plant leaf as input and classifies it into different disease categories using deep learning.”

🔹 2. Why did you use CNN?

👉 **Answer:**
“CNN is specifically designed for image processing. It automatically extracts features like edges, textures, and patterns, which is very useful for identifying plant diseases from images.”

🔹 3. What is Conv2D?

👉 **Answer:**
“Conv2D is a convolution layer that applies filters on the image to extract features like edges and patterns. It helps the model understand important visual information.”

🔹 4. What is the role of MaxPooling?

👉 **Answer:**
“MaxPooling reduces the size of feature maps and keeps only the important information. This makes computation faster and reduces overfitting.”

🔹 5. Why do we use ReLU activation?

👉 **Answer:**
“ReLU removes negative values and introduces non-linearity, which helps the model learn complex patterns efficiently.”

🔹 6. What is Flatten layer?

👉 **Answer:**
“Flatten converts 2D feature maps into a 1D vector so that it can be given to the fully connected Dense layer.”

🔹 **7. What is Dense layer?**

👉 **Answer:**
“Dense layer is a fully connected layer where each neuron is connected to all neurons from the previous layer. It performs final classification.”

🔹 **8. Why use Softmax in output layer?**

👉 **Answer:**
“Softmax converts outputs into probability values. It helps identify which class has the highest probability.”

🔹 **9. What is Dropout?**

👉 **Answer:**
“Dropout randomly disables some neurons during training to prevent overfitting and improve generalization.”

🔹 **10. What is overfitting?**

👉 **Answer:**
“Overfitting happens when the model learns training data too well but performs poorly on new data.”

🔹 **11. How did you prevent overfitting?**

👉 **Answer:**
“I used Dropout and data augmentation techniques like flipping, zooming, and shearing to improve generalization.”

🔹 **12. What is ImageDataGenerator?**

👉 **Answer:**
“It is used for preprocessing images and performing data augmentation like rescaling, flipping, and zooming.”

🔹 **13. Why rescale = 1./255?**

👉 **Answer:**
“Because pixel values range from 0 to 255, rescaling converts them to 0 to 1, which helps faster and stable training.”

🔹 **14. What is batch size?**

👉 **Answer:**
“Batch size is the number of images processed at one time during training. In my project, it is 32.”

🔹 **15. What is epoch?**

👉 **Answer:**
“Epoch means one complete pass of the entire training dataset through the model.”

🔹 **16. What optimizer did you use?**

👉 **Answer:**
“I used Adam optimizer because it is efficient and automatically adjusts learning rate.”

🔹 **17. What is loss function used?**

👉 **Answer:**
“I used categorical crossentropy because it is suitable for multi-class classification.”

🔹 **18. Difference between training and validation data?**

👉 **Answer:**
“Training data is used to train the model, while validation data is used to test model performance during training.”

🔹 **19. What is accuracy?**

👉 **Answer:**
“Accuracy is the percentage of correct predictions made by the model.”

🔹 **20. Why did you choose 128x128 image size?**

👉 **Answer:**
“It provides a good balance between computational efficiency and feature detail.”

🔹 **21. What happens if image size increases?**

👉 **Answer:**
“Accuracy may improve slightly, but computation time and memory usage will increase.”

🔹 **22. What is feature extraction?**

👉 **Answer:**
“It is the process of identifying important patterns like edges, textures, and shapes from images.”

🔹 **23. What is backpropagation?**

👉 **Answer:**
“It is a process where the model updates its weights based on error to improve accuracy.”

🔹 **24. Why use CNN instead of ANN?**

👉 **Answer:**
“CNN automatically extracts spatial features from images, while ANN cannot handle image data efficiently.”

🔹 **25. What is softmax output example?**

👉 **Answer:**
“For example: [0.1, 0.7, 0.2] → model predicts class with 70% probability.”

🔥 TRICKY QUESTIONS (VERY IMPORTANT)

🔹 26. What if dataset is small?

👉 **Answer:**
“We can use data augmentation or transfer learning to improve performance.”

🔹 27. What is transfer learning?

👉 **Answer:**
“It uses a pre-trained model like VGG16 or ResNet and fine-tunes it for our problem.”

🔹 28. How can you improve accuracy?

👉 **Answer:**

- Increase dataset
- Use deeper CNN
- Use transfer learning
- Tune hyperparameters

🔹 29. What are hyperparameters?

👉 **Answer:**
“Parameters like batch size, epochs, learning rate which are set before training.”

🔹 30. What is real-world application?

👉 **Answer:**
“It helps farmers detect diseases early and take proper action to improve crop yield.”